
Analyzing Inverse Problems with Invertible Neural Networks: Implementation and Experiments

Saksham Maitri

B.Tech. Math and Computing
Indian Institute of Science
sakshamm@iisc.ac.in

K. Sai Sandesh Reddy

B.Tech. Math and Computing
Indian Institute of Science
saisandeshk@iisc.ac.in

Yash Vijay Kamble

B.Tech. Math and Computing
Indian Institute of Science
yashvijay@iisc.ac.in

Harisrrinivas V.

B.Tech. Math and Computing
Indian Institute of Science
Harisrriniva@iisc.ac.in

Abstract

Inverse problems, determining hidden parameters (x) from observations (y), are prevalent but often ill-posed, necessitating estimation of the full posterior $p(x|y)$ to characterize ambiguity. Invertible Neural Networks (INNs) address this by learning the forward process $x \rightarrow [y, z]$, using latent variables z to retain lost information. Their bijective nature provides the inverse $[y, z] \rightarrow x$ implicitly, enabling efficient posterior sampling. This work implements and validates the INN architecture from Ardizzone et al. (2019) on Gaussian Mixture Model (GMM) and 2D Inverse Kinematics (IK) benchmarks. Experiments confirm the INN’s ability to capture multi-modal (GMM) and continuous (IK) ambiguities, demonstrating their utility for analyzing inverse problems.

1 Introduction

Inferring hidden system parameters x from indirect measurements y is a common challenge in science and engineering (1). While the forward process $s : x \rightarrow y$ is often known, the inverse problem $y \rightarrow x$ is frequently ill-posed: information loss means multiple x can yield the same y . A full understanding requires the posterior distribution $p(x|y)$, which captures ambiguities, quantifies uncertainty, and reveals degeneracies (1).

Traditional methods like direct regression struggle with ambiguity. Sampling techniques (MCMC, ABC) can be computationally expensive (8), while variational methods may use restrictive assumptions (9). Conditional generative models like cGANs might lack sample diversity (10).

Invertible Neural Networks (INNs) offer an alternative by learning the forward process (6; 2; 1). An INN maps inputs x to measurements y and auxiliary latent variables z : $f : x \rightarrow [y, z]$. The latent variables z , typically following a simple prior like $p(z) = \mathcal{N}(0, I)$, capture information about x not present in y . Crucially, f is bijective and efficiently invertible. The inverse $g = f^{-1} : [y, z] \rightarrow x$ is learned implicitly. To sample from the posterior $p(x|y^*)$ for an observation y^* , one samples $z \sim p(z)$ and computes $x = g(y^*, z)$ (1).

This work implements and evaluates the INN approach (1), replicating their Gaussian Mixture Model (GMM) and 2D Inverse Kinematics (IK) experiments. Our goal is to verify the implemented INN’s ability to learn the forward processes and accurately approximate the full posterior distributions, handling both multi-modality and continuous ambiguities.

2 Methodology

2.1 Foundation: Understanding Normalizing Flows via Real NVP

Before delving into the specific INN architecture for inverse problems, it is crucial to understand the underlying mechanism of normalizing flows. As prerequisite work, we successfully implemented and experimented with the Real NVP (Real Non-Volume Preserving) architecture (2), a prominent example of normalizing flows based on affine coupling layers. We applied this architecture to generative modeling tasks on standard datasets including MNIST, CIFAR-10, and synthetic 2D datasets from scikit-learn (specifically, the Two Moons and Two Concentric Circles datasets).

This hands-on implementation provided a strong foundation in the core principles: constructing bijective transformations using coupling layers, ensuring efficient computation of the forward pass, inverse pass, and the Jacobian determinant (essential for exact likelihood training via the change of variables formula), and understanding how stacking these layers can build complex, expressive density estimators. The architectural choices in Real NVP, particularly the use of masking and alternating coupling layers, directly inform the design of the INNs used in this project. This foundational understanding of normalizing flows is essential for applying similar invertible structures effectively to the challenges posed by inverse problems.

2.2 Problem Formulation and INN Framework

We aim to infer parameters $x \in \mathbb{R}^D$ from observations $y \in \mathbb{R}^M$ linked by $y = s(x)$, approximating the posterior $p(x|y)$. An INN f with parameters θ maps x to an augmented space $[y, z]$, where latent variables $z \in \mathbb{R}^K$ capture lost information, satisfying $D = M + K$ (potentially using padding).

$$[y, z] = f(x; \theta) = [f_y(x; \theta), f_z(x; \theta)] \quad (1)$$

We enforce a simple prior on latent variables, $z \sim p(z) = \mathcal{N}(z; 0, I)$. The INN’s invertibility guarantees the existence of $g = f^{-1}$:

$$x = g(y, z; \theta) \quad (2)$$

Sampling $z \sim p(z)$ and using $g(y^*, z)$ generates samples from the approximate posterior $q(x|y^*)$ for a given observation y^* .

2.3 INN Architecture

Leveraging the principles of normalizing flows, we employ coupling layers, specifically GLOW blocks (5) via the FrEIA library (3), similar to the RealNVP-based structure in (1). A coupling block splits input u into u_1, u_2 and applies invertible affine transformations:

$$v_1 = u_1 \odot \exp(s_2(u_2)) + t_2(u_2) \quad (3)$$

$$v_2 = u_2 \odot \exp(s_1(v_1)) + t_1(v_1) \quad (4)$$

where s_i, t_i are subnet functions (MLPs in our case) and $\odot$ is element-wise multiplication. The inverse is computed by reversing these steps. Stacking multiple blocks forms the INN. Random permutation layers are used between blocks for information mixing (1; 5). Zero-padding is applied if D is smaller than the network’s processing dimension to potentially enhance representational capacity.

2.4 Training

We use the bidirectional training strategy from (1).

Forward Loss (L_y): Enforces $f_y(x; \theta)$ to match ground truth $y = s(x)$, typically using Mean Squared Error (MSE) with potentially added noise for robustness.

$$L_y = \mathbb{E}_{x \sim p(x), y = s(x)} \|f_y(x; \theta) - y\|^2 \quad (5)$$

Latent Loss (L_z): Encourages $f_z(x)$ to follow the Gaussian prior $p(z)$ and be independent of y . Implemented either via distribution matching losses like Maximum Mean Discrepancy (MMD) or a simpler L2 penalty enforcing $z \approx 0$. For the Gaussian prior, an L2 penalty $\frac{1}{2} \|f_z(x; \theta)\|^2$ is often sufficient.

Backward Loss (L_x): Ensures the inverse map $g(y, z)$ generates samples consistent with the prior $p(x)$. Computed by sampling $z \sim p(z)$, using corresponding y , generating $x' = g(y, z)$, and comparing the distribution of x' to $p(x)$, often using MMD or a reconstruction loss.

Total Loss: A weighted sum of the individual losses guides training:

$$L_{\text{total}} = \lambda_y L_y + \lambda_z L_z + \lambda_x L_x \quad (6)$$

The weights $\lambda_y, \lambda_z, \lambda_x$ are hyperparameters balancing forward fit, latent space structure, and inverse consistency. We use the Adam optimizer (7). Specific loss choices and weights for GMM and IK are detailed in the Appendix.

3 Experiments

We first evaluated Normalizing Flows on MNIST and CIFAR-10 Dataset, and then we evaluated our INN implementation on the GMM and 2D IK benchmarks from (1), using PyTorch (4) and FrEIA (3). Detailed information about implementation details of Normalizing Flow experiments, and hyperparameters for INNs are in the Appendix.

3.1 Gaussian Mixture Model (GMM)

Task: Model the inverse mapping from a discrete class label y (one-hot) to 2D points x drawn from a specific subset of Gaussian mixture components. The goal is to sample from the correct multi-modal posterior $p(x|y^*)$ given a label y^* .

Implementation: Trained on 10^7 samples from 6 Gaussian components. Used an INN with 6 GLOW blocks and zero-padding. Bidirectional training employed MSE for L_y and multiscale MMD for L_z and L_x .

Results: Figure 1 compares ground truth (a) with INN posterior samples (b) and estimated classification regions (c). The INN accurately captures the multi-modal posteriors for each class label, generating samples from the correct components without mode collapse, consistent with (1).

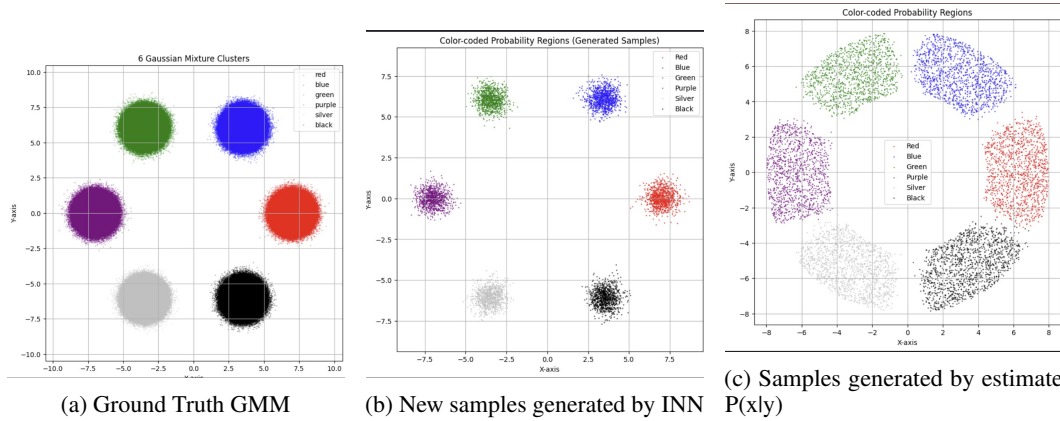


Figure 1: GMM results. (a) Ground truth. (b) Samples $x \sim q(x|y^*)$ from INN inverse. (c) INN-based classification generated by $\text{argmax}(\text{over } y)P(x|y)$.

3.2 Inverse Kinematics (IK)

Task: Infer the distribution of joint parameters x (1 prismatic, 3 rotational) for a 4-joint 2D arm, given a target end-effector position y^* . This involves handling continuous ambiguities.

Implementation: Used $N = 4$ joints, training on 10^6 samples with an INN of 6 GLOW blocks. Employed bidirectional training with specific loss weights emphasizing forward prediction and reverse reconstruction (details in Appendix).

Results: Figure 2 shows 100 arm configurations sampled from the learned posterior $q(x|y^*)$ for $y^* = [2, 0]$. The generated poses are diverse and place the end-effector (green circles) accurately

around the target (blue cross). The mean Euclidean distance between generated end-effectors and the target was 0.0552. This demonstrates the INN’s ability to capture the continuous solution manifold, matching the qualitative results in (1) (cf. Fig 3).

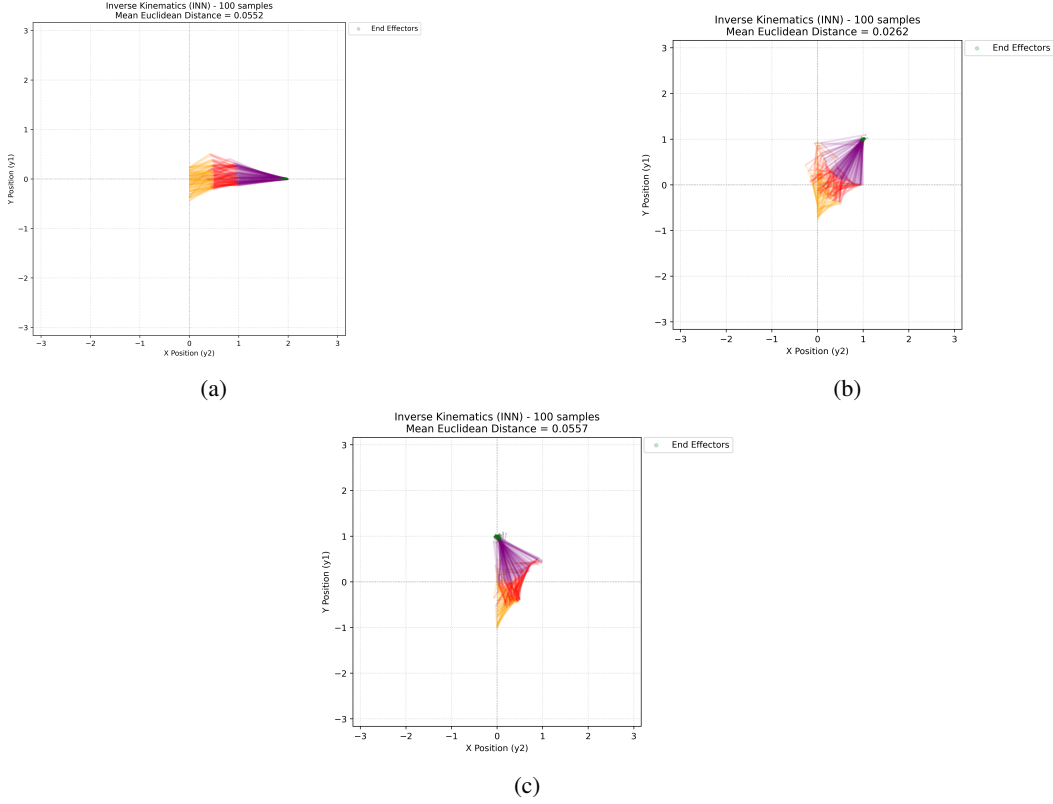


Figure 2

4 Conclusion

We successfully implemented and evaluated Invertible Neural Networks for inverse problems, replicating key results from (1). By training on the forward process and utilizing latent variables, our INNs effectively approximated the full posterior distribution $p(x|y)$ for both benchmark tasks.

The GMM experiment demonstrated the INN’s capability to capture discrete multi-modality without mode collapse. The IK experiment showed its proficiency in representing continuous ambiguities, generating diverse and valid solutions. These findings validate the utility of INNs as a powerful tool for analyzing ill-posed inverse problems, offering a principled way to perform posterior inference and quantify uncertainty. INNs provide a computationally efficient alternative to traditional methods, granting access to the full posterior distribution essential for understanding complex systems.

References

1. L. Ardizzone, J. Kruse, S. Wirkert, D. Rahner, E. W. Pellegrini, R. S. Klessen, L. Maier-Hein, C. Rother, and U. Köthe. Analyzing inverse problems with invertible neural networks. In *International Conference on Learning Representations (ICLR)*, 2019. URL <https://openreview.net/forum?id=H1g9osA5FX>.
2. L. Dinh, J. Sohl-Dickstein, and S. Bengio. Density estimation using Real NVP. In *International Conference on Learning Representations (ICLR)*, 2017.
3. L. Ardizzone, J. Kruse, R. Mackowiak, C. Rother, and U. Köthe. FrEIA: A Framework for Easily Invertible Architectures. Zenodo, Oct 2018. doi: 10.5281/zenodo.2539711. URL <https://doi.org/10.5281/zenodo.2539711>.

4. A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems*, pp. 8026–8037, 2019.
5. D. P. Kingma and P. Dhariwal. Glow: Generative flow with invertible 1x1 convolutions. In *Advances in Neural Information Processing Systems*, pp. 10215–10224, 2018.
6. L. Dinh, D. Krueger, and Y. Bengio. NICE: Non-linear independent components estimation. In *International Conference on Learning Representations (ICLR) workshop track*, 2015.
7. D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
8. M. Sunnåker, A. G. Busetto, E. Numminen, J. Corander, M. Foll, and C. Dessimoz. Approximate Bayesian computation. *PLoS computational biology*, 9(1):e1002803, 2013. Public Library of Science.
9. D. M. Blei, A. Kucukelbir, and J. D. McAuliffe. Variational inference: A review for statisticians. *Journal of the American statistical Association*, 112(518):859–877, 2017. Taylor & Francis.
10. J.-Y. Zhu, R. Zhang, D. Pathak, T. Darrell, A. A. Efros, O. Wang, and E. Shechtman. Toward multimodal image-to-image translation. In *Advances in neural information processing systems*, pp. 465–476, 2017.
11. A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola. A kernel two-sample test. *Journal of Machine Learning Research*, 13(Mar):723–773, 2012.
12. Y. Song, C. Meng, and S. Ermon. MintNet: Building Invertible Neural Networks with Masked Convolutions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

A Appendix

A.1 Theoretical Foundations

A.1.1 Proof of Invertibility from Non-Singular Jacobian

This section outlines the proof (adapted from (12) Appendix A) that a function f belonging to a class $\mathcal{F}$ of continuously differentiable functions with (lower) triangular Jacobians is invertible if its Jacobian $J_f(x)$ is non-singular everywhere in the domain.

Lemma A.1 (Dependency Structure - Lemma 1 in (12)). *If the Jacobian $J_f(x)$ of a function $f : \mathbb{R}^D \rightarrow \mathbb{R}^D$ is lower triangular for all $x \in \mathbb{R}^D$, then the i -th component $[f(x)]_i$ is a function of only the first i input variables, $x_1, \dots, x_i$, and does not depend on $x_{i+1}, \dots, x_D$.*

Proof. The fact that $J_f(x)$ is lower triangular means that $[J_f(x)]_{ij} = \frac{\partial [f(x)]_i}{\partial x_j} = 0$ for all $j > i$. Consider the value of $[f(x)]_i$ when only x_j varies (for $j > i$) and all other components x_k ($k \neq j$) are held constant. By the fundamental theorem of calculus:

$$[f(x_1, \dots, x_j, \dots, x_D)]_i - [f(x_1, \dots, 0, \dots, x_D)]_i = \int_0^{x_j} \frac{\partial [f(x_1, \dots, t, \dots, x_D)]_i}{\partial t} dt \quad (7)$$

$$= \int_0^{x_j} [J_f(x_1, \dots, t, \dots, x_D)]_{ij} dt \quad (8)$$

Since $j > i$, the integrand $[J_f(\dots)]_{ij}$ is always 0. Thus, the integral is 0, implying that $[f(x)]_i$ does not depend on x_j for any $j > i$. $\square$

Lemma A.2 (Monotonicity - Lemma 2 in (12)). *If the product of diagonal Jacobian elements $\text{diag}(J_f(x)J_f(0))$ is strictly positive for all $x \in \mathbb{R}^D$, then for any $i \in \{1, \dots, D\}$, the diagonal element $[J_f(x)]_{ii}$ maintains a constant sign (either always positive or always negative) for all x . Consequently, if f is continuously differentiable and $J_f(x)$ is lower triangular, then $[f(x)]_i$ is strictly monotonic with respect to x_i when $x_1, \dots, x_{i-1}$ are held fixed.*

Proof. The condition $\text{diag}(J_f(x)J_f(0)) > 0$ means $[J_f(x)]_{ii}[J_f(0)]_{ii} > 0$ for all i and all x . This implies that $[J_f(x)]_{ii}$ never changes sign and is never zero, because if it did change sign, by the Intermediate Value Theorem (since f is continuously differentiable, J_f is continuous), it would have to pass through zero at some point, contradicting the condition. Thus, $[J_f(x)]_{ii}$ has the same sign as the constant $[J_f(0)]_{ii}$ for all x . The partial derivative of $[f(x)]_i$ with respect to x_i , keeping $x_1, \dots, x_{i-1}$ fixed (and noting from Lemma A.1 that $[f(x)]_i$ does not depend on $x_{i+1}, \dots, x_D$), is precisely $[J_f(x)]_{ii}$. Since $[J_f(x)]_{ii}$ is always non-zero and maintains a constant sign, $[f(x)]_i$ is strictly monotonic with respect to x_i . $\square$

Theorem A.3 (Invertibility - Theorem 1 in (12)). *If $f \in \mathcal{F}$ (i.e., f is continuously differentiable and $J_f(x)$ is lower triangular) and $J_f(x)$ is non-singular for all x in the domain, then f is invertible.*

Proof. First, we show that the condition $\text{diag}(J_f(x)J_f(0)) > 0$ holds. Since $J_f(x)$ is non-singular and triangular, its diagonal elements $[J_f(x)]_{ii}$ must be non-zero for all x and i . Assume, for contradiction, that for some i and x' , $[J_f(x')]_{ii}[J_f(0)]_{ii} \leq 0$. Since neither term is zero, we must have $[J_f(x')]_{ii}[J_f(0)]_{ii} < 0$, meaning they have opposite signs. Consider $h(t) = [J_f(tx')]_{ii}$ for $t \in [0, 1]$. Since $h(0)$ and $h(1)$ have opposite signs and h is continuous, the Intermediate Value Theorem implies there exists $t^* \in (0, 1)$ such that $h(t^*) = [J_f(t^*x')]_{ii} = 0$. This contradicts the non-singularity of $J_f(t^*x')$. Thus, $\text{diag}(J_f(x)J_f(0)) > 0$ holds.

Next, we prove existence and uniqueness of x such that $f(x) = z$ for any z in the range of f . We solve for $x_1, \dots, x_D$ sequentially. For $i = 1$: By Lemma A.1, $[f(x)]_1$ depends only on x_1 . By Lemma A.2 (using the result just proved), $[f(x)]_1$ is strictly monotonic in x_1 . Thus, the equation $[f(x)]_1 = z_1$ has a unique solution for x_1 for any z_1 in the range of $[f]_1$. Assume we have uniquely found $x_1, \dots, x_k$. For $i = k + 1$: By Lemma A.1, $[f(x)]_{k+1}$ depends only on $x_1, \dots, x_{k+1}$. Since $x_1, \dots, x_k$ are fixed, the equation $[f(x)]_{k+1} = z_{k+1}$ is an equation for x_{k+1} only. By Lemma A.2, $[f(x)]_{k+1}$ is strictly monotonic in x_{k+1} (given fixed $x_1, \dots, x_k$). Therefore, $[f(x)]_{k+1} = z_{k+1}$ has a unique solution for x_{k+1} for any valid z_{k+1} .

By induction, we can uniquely determine $x_1, \dots, x_D$ for any z . This shows $f^{-1}(z)$ exists and is unique. $\square$

A.1.2 Proof of Correctness of Generated Posteriors

This section reproduces the theoretical justification for why the INN approach recovers the true posterior under ideal conditions.

Lemma A.4. *If some bijective function $f : x \rightarrow z$ transforms a probability density $p_x(x)$ to $p_z(z)$, then the inverse function f^{-1} transforms $p_z(z)$ back to $p_x(x)$.*

Proof. Let $p'_x(x)$ be the density obtained by transforming $p_z(z)$ back using f^{-1} . By the change of variables formula:

$$\text{Forward: } p_z(z) = p_x(x = f^{-1}(z)) |\det[J_{f^{-1}}(z)]| \quad (9)$$

$$\text{Reverse: } p'_x(x) = p_z(z = f(x)) |\det[J_f(x)]| \quad (10)$$

where $J_f(x)$ is the Jacobian of f at x , and $J_{f^{-1}}(z)$ is the Jacobian of f^{-1} at z . Substituting Eq. (9) into Eq. (10):

$$p'_x(x) = [p_x(x = f^{-1}(f(x))) |\det[J_{f^{-1}}(f(x))]|] |\det[J_f(x)]| \quad (11)$$

$$= p_x(x) |\det[J_{f^{-1}}(f(x))]| \cdot |\det[J_f(x)]| \quad (12)$$

$$= p_x(x) |\det[J_{f^{-1}}(f(x)) \cdot J_f(x)]| \quad (13)$$

$$= p_x(x) |\det[I]| = p_x(x) \quad (14)$$

Here we used $f^{-1}(f(x)) = x$ and the chain rule property that the Jacobian of the inverse is the inverse of the Jacobian: $J_{f^{-1}}(f(x)) = [J_f(x)]^{-1}$. $\square$

Theorem A.5. *If an INN $f(x) = [y, z]$ is trained such that the supervised loss $L_y = \mathbb{E}[(y - f_y(x))^2]$ and the unsupervised latent loss $L_z = D(q(y, z), p(y)p(z))$ both reach zero (where D is a distance metric like MMD), then sampling $x = g(y^*, z)$ with $g = f^{-1}$ and $z \sim p(z)$ returns samples from the true posterior $p(x|y^*)$ for any measurement y^* .*

Proof. Let $p_z(z)$ be the target latent distribution and $p_y(y)$ be the marginal distribution of observations. Let $q(y, z)$ be the joint distribution of the network outputs when the input x is drawn from its prior $p_x(x)$. If $L_z = D(q(y, z), p_y(y)p_z(z)) = 0$, assuming D is a proper metric (like MMD (11)), then the output distribution matches the target product of marginals:

$$q(y, z) = p_y(y)p_z(z) \quad (15)$$

This implies that the generated y and z outputs are independent when $x \sim p_x(x)$.

Now, consider transforming the true conditional posterior $p(x|y^*)$ using the perfectly trained forward network f . Let the resulting output distribution be $q^*(y, z)$. Since $L_y = \mathbb{E}[(s(x) - f_y(x))^2] = 0$, the forward pass must perfectly reproduce the observation corresponding to any input x . For all x sampled from $p(x|y^*)$, the corresponding observation is $y = s(x) = y^*$. Therefore, the y component of the output distribution must be a Dirac delta centered at y^* : $q^*(y) = \delta(y - y^*)$. Because the overall output distribution $q(y, z)$ exhibits independence between y and z (Eq. (15)) when averaged over $x \sim p_x(x)$, and this property holds for the conditional case as well (assuming standard INN constructions), the z component of the conditional output distribution $q^*(z)$ must still follow the prior $p_z(z)$. Thus, the joint distribution of outputs when transforming $p(x|y^*)$ is:

$$q^*(y, z) = \delta(y - y^*)p_z(z) \quad (16)$$

Now, consider the inverse process: sampling $z \sim p_z(z)$ and feeding the pair $[y^*, z]$ into the inverse network $g = f^{-1}$. This procedure is equivalent to sampling $[y, z]$ from the distribution $q^*(y, z)$ defined in Eq. (16). By Lemma A.4, since f transforms $p(x|y^*)$ to $q^*(y, z)$, the inverse function $g = f^{-1}$ must transform $q^*(y, z)$ back to $p(x|y^*)$. Therefore, sampling via $x = g(y^*, z)$ with $z \sim p_z(z)$ yields samples from the true posterior $p(x|y^*)$. $\square$

Corollary A.6. *If the conditions of Theorem A.5 are met, the unsupervised reverse loss $L_x = D(q(x), p_x(x))$ between the marginalized distribution of inverted samples $q(x)$ and the prior data distribution $p_x(x)$ will also be 0.*

Proof. From Theorem A.5, the INN generates samples from the correct posterior $q(x|y^*) = p(x|y^*)$ for any y^* . The marginalized distribution $q(x)$ is obtained by averaging over y^* drawn from the data distribution $p_y(y^*)$:

$$q(x) = \int q(x|y^*)p_y(y^*)dy^* \quad (17)$$

$$= \int p(x|y^*)p_y(y^*)dy^* \quad (18)$$

$$= \int \frac{p(y^*|x)p_x(x)}{p_y(y^*)}p_y(y^*)dy^* \quad (\text{Bayes' Theorem}) \quad (19)$$

$$= p_x(x) \int p(y^*|x)dy^* \quad (20)$$

$$= p_x(x) \cdot 1 = p_x(x) \quad (21)$$

Since $q(x) = p_x(x)$, the distance $L_x = D(q(x), p_x(x))$ must be 0 if D is a proper metric. This justifies using L_x to potentially accelerate convergence without changing the theoretical optimum where $L_y = 0$ and $L_z = 0$. $\square$

A.2 Contributions

All team members contributed to the literature review, final discussions, and collaborative refinement of the project report and presentation. Specific individual contributions include:

- **K. Sai Sandesh Reddy:**
 - Contributed significantly to the foundational *Normalizing Flow (RealNVP)* implementation. (A.3)
 - Focused on coding and verifying the core components, including the affine coupling layers and the multi-scale architecture for image data (MNIST, CIFAR-10).
 - Set up and managed the training pipelines, including implementing the NLL loss and optimization procedures.
 - Ran key experiments on the benchmark datasets and was involved in analyzing the quantitative BPD results.
 - Contributed to the testing of INN on toy datasets and robotic arm experiments.
- **Harisrrinivas V.:**
 - Played a key role in the implementation and experimentation phase for the core *Normalizing Flow (RealNVP)* models.
 - Focused on implementing and debugging the data preprocessing (logit transform) and visualization aspects, particularly the 2D flow animations and the MNIST latent space interpolation tool.
 - Assisted in running experiments across different datasets and analyzing the qualitative results (visualizations).
 - Contributed to the literature review, report, creation of video, and refinement of the presentation narrative.
 - Also contributed to drafting the methodology section of the report and preparing presentation slides.
- **Yash Vijay Kamble:**
 - Implemented the *Inverse Kinematics* application using the Invertible Neural Network (INN) framework.
 - Adapted the core RealNVP architecture to handle the mapping between arm parameters (x), end-effector position (y), and latent variables (z).
 - Implemented and tuned the forward (L_y regression loss) and backward (MMD-based L_z, L_x) loss components specific to the INN training for this problem.
 - Developed the visualization component to effectively display multiple valid arm configurations (posterior samples $P(x|y^*)$) corresponding to a target end-effector position y^* .

- Contributed significantly to writing the methodology and experiment sections of the final report.
- Played a key role in creating the video demonstration.
- **Saksham Maitri:**
 - Implemented the *Gaussian Mixture Model (GMM)* inverse problem using the INN framework.
 - Trained the GMM INN model and generated visualizations showcasing its ability to capture multi-modal posterior distributions $p(x|y^*)$.
 - Provided an additional implementation and analysis for the *Inverse Kinematics* problem, contributing to the understanding of the INN’s behavior on this task.
 - Contributed to the experimental results analysis in the report.

A.3 Normalizing Flow Implementation and Analysis

To build the necessary foundation for tackling inverse problems using INNs (?), we undertook a thorough implementation and analysis of the RealNVP (?) normalizing flow architecture. This involved testing on low-dimensional toy datasets for conceptual clarity and scaling up to standard image benchmarks for evaluating performance and latent space properties.

2D Toy Datasets (Moons, Circles) To gain fundamental insights and visually inspect the flow dynamics, a RealNVP model specifically designed for 2D data was developed.

- **Core Architecture (RealNVP_2D):**
 - *Coupling Layers:* The model is constructed as a sequence of "Affine Coupling Layer" modules. Each layer partitions the 2D input x based on a mask and applies an affine transformation $y = \text{mask} * x + (1 - \text{mask}) * (x * \exp(s) + t)$ to one partition, conditioned on the other.
 - *Scale and Translation Networks (s/t):* Small Multi-Layer Perceptrons (MLPs) compute the scale s and translation t parameters based on the non-transformed partition of the input. The scale output is passed through $\tanh$ and scaled by a learnable parameter for stability. Note: Here we don’t need complex networks for s and t as the dataset is not complex
 - *Masking Strategy:* Alternating 'checkerboard' and 'channel' masks are used in successive layers. This ensures that after multiple layers, each input dimension has influenced each output dimension.
 - *Invertibility and Jacobian:* The architecture is invertible by above theorem, and the logarithm of the Jacobian determinant is efficiently computed by summing the s terms from the transformed partitions across all layers.
- **Training:** The model was trained by minimizing the Negative Log-Likelihood (NLL), using the standard Gaussian $p_Z(z)$ as the prior and the Adam optimizer.
- **Visualizations Created:** We generated detailed animations for the 2D datasets. Frames captured during training illustrate the learned transformation, showing:
 - (1) Original data x
 - (2) Data mapped to latent space $z = f(x)$ with grid deformation
 - (3) Prior samples z_{sample}
 - (4) Generated data $x_{\text{gen}} = f^{-1}(z_{\text{sample}})$ with inverse grid deformation.

MNIST Dataset (1x28x28) To evaluate scalability, a multi-scale RealNVP architecture was implemented for the grayscale MNIST dataset.

- **Core Architecture (RealNVP - MNIST Adaptation):**
 - *Coupling Layers:* Affine coupling layers form the basic building blocks within each scale.
 - *Scale and Translation Networks (s/t):* Standard ResNet blocks, including Conv2d, Batch-Norm2d, and ReLU activations, were used to compute the s and t parameters, providing greater expressive power for image data.

- **Multi-Scale Processing:** The architecture employed `num_scales=2` stages.
 - **Input Padding:** The single input channel was padded to two channels (`PadChannel`) to facilitate channel splitting.
 - **Squeeze Operation:** `nn.PixelUnshuffle(2)` was used to decrease spatial resolution ($28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7$) while increasing channel depth ($2 \rightarrow 8 \rightarrow 32$). Spatial padding (`nn.ConstantPad2d`) was applied before squeezing when dimensions were odd.
 - **Factor-Out:** At each scale transition (except the last), half the channels were split off and directly contributed to the final latent variable z . The remaining channels were passed to the next scale.
- **Masking:** Alternating checkerboard and channel masks were applied within the coupling layers at each scale.
- **Preprocessing:** An invertible logit transform ($\alpha = 0.05$) combined with uniform dequantization was applied to map pixel values from the bounded $[0, 1]$ range to an unbounded domain.
- **Training:** The model was trained by minimizing the Negative Log-Likelihood (NLL), using the standard Gaussian $p_Z(z)$ as the prior and the Adam optimizer.
- **Quantitative Result:** The model achieved **1.59 BPD** on the MNIST test set.
- **Visualization Created:** We developed an interactive tool specifically for MNIST which allows visualizing the manifold structure by performing linear interpolation between the latent codes (z_1, z_2) of user-selected start and end digit images (x_1, x_2).

CIFAR-10 Dataset (3x32x32) The multi-scale RealNVP architecture was further adapted and tested on the more complex CIFAR-10 dataset.

- **Core Architecture (RealNVP - CIFAR-10 Adaptation):**
 - The overall multi-scale structure (squeeze, factor-out) and coupling layer mechanism were similar to the MNIST adaptation but adjusted for the 3 input channels and 32×32 spatial resolution.
 - **Scale and Translation Networks (s/t):** ResNet blocks (including Conv2d, BatchNorm2d, ReLU) were again used to compute the affine parameters s and t within the coupling layers.
 - Preprocessing used the same logit transform approach.
 - **Training:** The model was trained by minimizing the Negative Log-Likelihood (NLL), using the standard Gaussian $p_Z(z)$ as the prior and the Adam optimizer.
 - **Quantitative Result:** The model achieved **3.78 BPD** on the CIFAR-10 test set. This result is reasonably close to the performance reported in the original RealNVP paper (3.49 BPD).

Conclusion Our RealNVP implementation successfully learned complex, high-dimensional, invertible transformations, validated by quantitative results on benchmarks and qualitative visualizations (2D, MNIST) confirming effective density modeling and structured latent spaces. While performance could be further optimized with more training (limited by computational resources), this robust implementation provides the necessary foundation for applying INNs to inverse problem analysis (1).

A.4 INN Implementation Details and Hyperparameters

- **Software:** Python, PyTorch (4), FrEIA (3).
- **Hardware:** Experiments were run on the Nvidia RTX 4090, 3070 and local machines of KIAC Lab.
- **INN Architecture:** Both experiments used INNs composed of 6 GLOW coupling blocks (5) with random permutations between blocks. Subnets within coupling blocks were MLPs with LeakyReLU/ReLU activations.
- **Gaussian Mixture Model (GMM):**
 - Data: 10^7 training samples from a mixture of 6 2D Gaussians.
 - Input dimension $D = 2$, Output label dimension $M = 6$ (one-hot).

- INN processing dimension: 16 (input x padded with 14 noise dimensions, output y padded with 10 noise dimensions). Latent dimension $K = 16 - 6 = 10$.
- Subnet hidden units: 512.
- Losses: L_y : MSE. L_z : Multiscale MMD (Rational Quadratic kernels) comparing $f_z(x)$ to $\mathcal{N}(0, I)$. L_x : Multiscale MMD comparing $g(y, z)$ (with y from data, $z \sim \mathcal{N}(0, I)$) to prior data $x \sim p(x)$.
- Training: 100 epochs, batch size 8192, Adam optimizer, learning rate schedule applied (details in code).

• **Inverse Kinematics (IK):**

- Data: 10^6 training samples. 4-joint arm (1 prismatic, 3 rotational).
- Input dimension $D = 4$, Output dimension $M = 2$ (end-effector position). Latent dimension $K = D - M = 2$.
- Arm lengths: $L = [0.5, 0.5, 1.0]$ (for rotational joints). Joint priors: $x_i \sim \mathcal{N}(0, \sigma_i^2)$ with $\sigma = [0.25, 0.5, 0.5, 0.5]$.
- Data Generation Process:
 - * **Sampling Joint Configurations:** Joint parameters $x = [x_1, x_2, x_3, x_4]$ are randomly drawn from $x_i \sim \mathcal{N}(0, \sigma_i^2)$, with $\sigma = [0.25, 0.5, 0.5, 0.5]$. 10^6 samples are generated and processed in batches.
 - * **Computing End Effector Positions:** The forward kinematics function $s(x)$ maps x to $y = [y_x, y_y]$ iteratively:

$$p_0 = [0, x_1]^T \quad (\text{Base position with vertical translation})$$

$$\theta_1 = x_2$$

$$p_1 = p_0 + L_1 [\cos(\theta_1), \sin(\theta_1)]^T$$

$$\theta_2 = \theta_1 + x_3$$

$$p_2 = p_1 + L_2 [\cos(\theta_2), \sin(\theta_2)]^T$$

$$\theta_3 = \theta_2 + x_4$$

$$y = p_3 = p_2 + L_3 [\cos(\theta_3), \sin(\theta_3)]^T$$

where $L = [0.5, 0.5, 1.0]$, and θ_i are cumulative angles.

- Determining Joint Parameters (θ) from (x, y) :
 - * Given a target (x, y) , the INN inverts the mapping using $g(y, z; \theta)$ with random $z \sim \mathcal{N}(0, I)$.
 - * Multiple solutions (e.g., 100) are generated by sampling different z values, exploring diverse θ configurations.
- Losses: The loss functions guiding the INN training are:

$$L_y = \text{MSE}(f_y(x; \theta), s(x))$$

$$L_z = \frac{1}{2} \|f_z(x; \theta)\|^2$$

$$L_x = \text{MSE}(s(g(y, z; \theta)), y) + \text{MSE}(g(y', z; \theta), x)$$

$$L = \lambda_y L_y + \lambda_z L_z + \lambda_x L_x$$

where:

- * $f_y(x; \theta)$ and $f_z(x; \theta)$ are the position and latent outputs.
- * $g(y, z; \theta)$ is the inverse mapping.
- * $s(\cdot)$ is the forward kinematics function.
- * Loss weights: $\lambda_y = 50.0$, $\lambda_z = 0.5$, $\lambda_x = 100.0$.

- Training: 50 epochs, Adam optimizer, learning rate schedule applied. Target position for evaluation: $y^* = [2, 0]$.

- **Loss Variations:** Note the difference in the implementation of L_z and L_x between the GMM (MMD-based) and IK (L2-penalty/reconstruction-based) experiments, reflecting different strategies to enforce latent structure and inverse consistency.

- **Link to Repo(s):** <https://github.com/IISc-UMC-203>